aula9

April 24, 2026

1 Linguagens e Ambientes de Programação

1.1 (Aula Teórica 9)

1.1.1 Agenda

- Exercícios sobre listas
- Funções de ordem superior
- Funções map e fold.

2 Lista de exemplos

1. Iterar listas de inteiros
    1. encontrar um elemento
    2. calcular a média dos elementos da lista
    3. contar os elementos maiores que um valor
    4. contar os elementos dado um critério (ordem superior)
2. Transformar listas
    1. projeção
    2. filtro
    3. Listas e pares: zip e unzip
    4. composição de funções
3. Compactar elementos
    1. Descompactar
4. Agrupar elementos
    1. Desagrupar elementos
5. merge de duas listas ordenadas
6. merge sort

3 Iterar listas de inteiros

3.1 Encontrar um elemento

[79]:
```
(** [mem a l] is true if and only if a is equal to an element of list [l].
    pre: [true]
*)
let rec mem x l =
    match l with
```

```
    | [] -> false
    | y::ys -> if x = y then true else mem x ys
```

[79]:
```
val mem : 'a -> 'a list -> bool = <fun>
```

[80]:
```
let _ = assert (mem 1 [1;2;3;4;5] = true)
let _ = assert (mem 'a' ['a';'b';'c';'d';'e'] = true)
```

[80]:
```
- : unit = ()
```

[80]:
```
- : unit = ()
```

3.2 Calcular a média dos elementos da lista

[81]:
```
(** [avg l] is the average of the elements in list [l].
    pre: [length l > 0] is not empty.
    *)
let avg l =
    let rec sum l =
        match l with
        | [] -> 0
        | x::xs -> x + sum xs
    in
    let rec len l =
        match l with
        | [] -> 0
        | x::xs -> 1 + len xs
    in
    (sum l) / (len l)
```

[81]:
```
val avg : int list -> int = <fun>
```

[45]:
```
(** [avg l] is the average of the elements in list [l].
    pre: [length l > 0] is not empty.
    *)
let avg l =
  let rec sum_count l =
    match l with
    | [] -> (0, 0)
    | x::xs ->
      let (sum, count) = sum_count xs in
      (sum + x, count + 1)
  in
  let (sum, count) = sum_count l in sum / count
```

[45]:
```
val avg : int list -> int = <fun>
```

[48]:
```
(** [avg l] is the average of the elements in list [l].
    pre: [length l > 0] is not empty.
    *)
let avg l =
  let rec sum_count l acc count =
    match l with
    | [] -> (acc, count)
    | x::xs -> sum_count xs (acc + x) (count + 1)
  in
  let (sum, count) = sum_count l 0 0 in sum / count
```

[48]:
```
val avg : int list -> int = <fun>
```

[54]:
```
(** [avg l] is the average of the elements in list [l].
    pre: [length l > 0] is not empty.
    *)
let avg l =
  let (sum, count) = List.fold_left (fun (sum, count) x -> (sum + x, count +␣
 ↪1)) (0, 0) l
  in sum/count
```

[54]:
```
val avg : int list -> int = <fun>
```

[49]:
```
let _ = assert (avg [1;2;3;4;5] = 3)
let _ = assert (avg [1;2;3;4;5;6;7] = 4)
```

[49]:
```
- : unit = ()
```

[49]:
```
- : unit = ()
```

3.3 Contar os elementos maiores que um valor

[1]:
```
(** [count_greater_than x l] is the number of elements in list [l] that are␣
 ↪greater than [x].
    pre: [true]
    *)
    let rec count_greater_than x l =
      match l with
      | [] -> 0
      | y::ys -> if x < y
                 then 1+count_greater_than x ys
```

```
                  else count_greater_than x ys
```

[1]:
```
val count_greater_than : 'a -> 'a list -> int = <fun>
```

[82]:
```
(** [count_greater_than x l] is the number of elements in list [l] that are␣
 ↪greater than [x].
    pre: [true]
    *)
let rec count_greater_than x l =
    match l with
    | [] -> 0
    | y::ys ->
      if y > x then 1 + count_greater_than x ys
      else count_greater_than x ys
```

[82]:
```
val count_greater_than : 'a -> 'a list -> int = <fun>
```

[2]:
```
let _ = assert (count_greater_than 3 [1;2;3;4;5] = 2)
let _ = assert (count_greater_than 'f' ['a';'b';'c';'d';'e';'f';'g'] = 1)
let _ = assert (count_greater_than 3. [1.;2.;3.;4.;5.] = 2)
let _ = assert (count_greater_than "hello" ["hello";"world";"hello";"world";
 ↪"hello"] = 2)
```

[2]:
```
- : unit = ()
```

[2]:
```
- : unit = ()
```

[2]:
```
- : unit = ()
```

[2]:
```
- : unit = ()
```

[3]:
```
(** [count_greater_than x l] is the number of elements in list [l] that are␣
 ↪greater than [x].
    pre: [true]
    *)
let count_greater_than x l =
  List.fold_left (fun n y -> if y > x then n + 1 else n) 0 l
```

[3]:
```
val count_greater_than : 'a -> 'a list -> int = <fun>
```

[4]:
```
let _ = assert (count_greater_than 3 [1;2;3;4;5] = 2)
let _ = assert (count_greater_than 'f' ['a';'b';'c';'d';'e';'f';'g'] = 1)
let _ = assert (count_greater_than 3. [1.;2.;3.;4.;5.] = 2)
let _ = assert (count_greater_than "hello" ["hello";"world";"hello";"world";
 ↪"hello"] = 2)
```

[4]:
```
- : unit = ()
```

[4]:
```
- : unit = ()
```

[4]:
```
- : unit = ()
```

[4]:
```
- : unit = ()
```

3.4 Contar os elementos dado um critério (ordem superior)

[5]:
```
(** [count_if p l] is the number of elements in list [l] that satisfy predicate␣
 ↪[p].
    pre: [true]
    *)
let rec count_if p l =
    match l with
    | [] -> 0
    | x::xs ->
      if p x then 1 + count_if p xs
      else count_if p xs
```

[5]:
```
val count_if : ('a -> bool) -> 'a list -> int = <fun>
```

[7]:
```
let count_greater_than n = count_if (fun x -> x > n)

let _ = assert (count_greater_than 3 [1;2;3;4;5] = 2)
let _ = assert (count_greater_than 'f' ['a';'b';'c';'d';'e';'f';'g'] = 1)
let _ = assert (count_greater_than 3. [1.;2.;3.;4.;5.] = 2)
let _ = assert (count_greater_than "hello" ["hello";"world";"hello";"world";
 ↪"hello"] = 2)
```

[7]:
```
val count_greater_than : 'a -> 'a list -> int = <fun>
```

[7]:
```
- : unit = ()
```

[7]: - : unit = ()

[7]: - : unit = ()

[7]: - : unit = ()

[55]:
```
let count_less_than n = count_if (fun x -> x < n)

let _ = assert (count_less_than 3 [1;2;3;4;5] = 2)
let _ = assert (count_less_than 'f' ['a';'b';'c';'d';'e';'f';'g'] = 5)
let _ = assert (count_less_than 3. [1.;2.;3.;4.;5.] = 2)
let _ = assert (count_less_than "hello" ["hello";"world";"hello";"world";
 ↪"hello"] = 0)
```

[55]: val count_less_than : 'a -> 'a list -> int = <fun>

[55]: - : unit = ()

[55]: - : unit = ()

[55]: - : unit = ()

[55]: - : unit = ()

[8]:
```
let count_pairs_wtv = count_if (fun (a,b) -> a > b)
```

[8]: val count_pairs_wtv : ('_weak1 * '_weak1) list -> int = <fun>

[9]:
```
type person = {name: string; age: int}
let count_age_equal_to n = count_if (fun x -> x.age = n)

let _ = assert (count_age_equal_to 3 [{name="a"; age=1}; {name="b"; age=2};␣
 ↪{name="c"; age=3}; {name="d"; age=4}; {name="e"; age=3}] = 2)
let _ = assert (count_age_equal_to 1 [{name="a"; age=1}; {name="b"; age=2};␣
 ↪{name="c"; age=3}; {name="d"; age=4}; {name="e"; age=5}] = 1)
```

[9]: type person = { name : string; age : int; }

```
[9]: val count_age_equal_to : int -> person list -> int = <fun>

[9]: - : unit = ()

[9]: - : unit = ()
```

```
[10]: (** [count_if p l] is the number of elements in list [l] that satisfy predicate␣
       ↪[p].
          pre: [true]
          *)
      let count_if p l =
        List.fold_left (fun n x -> if p x then n + 1 else n) 0 l

[10]: val count_if : ('a -> bool) -> 'a list -> int = <fun>
```

```
[11]: type person = {name: string; age: int}
      let count_age_equal_to n = count_if (fun x -> x.age = n)

      let _ = assert (count_age_equal_to 3 [{name="a"; age=1}; {name="b"; age=2};␣
       ↪{name="c"; age=3}; {name="d"; age=4}; {name="e"; age=3}] = 2)
      let _ = assert (count_age_equal_to 1 [{name="a"; age=1}; {name="b"; age=2};␣
       ↪{name="c"; age=3}; {name="d"; age=4}; {name="e"; age=5}] = 1)

[11]: type person = { name : string; age : int; }

[11]: val count_age_equal_to : int -> person list -> int = <fun>

[11]: - : unit = ()

[11]: - : unit = ()
```

4 Transformar listas

4.1 Projeção

```
[12]: (** [project_fst l] is the list of the first elements of the pairs in list [l].
          pre: [true]
          *)
      let rec project_fst l =
        match l with
```

```
  | [] -> []
  | (x,_)::xs -> x::project_fst xs
```

[12]: val project_fst : ('a * 'b) list -> 'a list = <fun>

[13]:
```
let _ = assert (project_fst [] = [])
let _ = assert (project_fst [(1,2);(3,4);(5,6)] = [1;3;5])
```

[13]: - : unit = ()

[13]: - : unit = ()

[14]:
```
(** [project_fst l] is the list of the first elements of the pairs in list [l].
    pre: [true]
    *)
let project_fst l =
  List.map (fun (x,_) -> x) l
```

[14]: val project_fst : ('a * 'b) list -> 'a list = <fun>

[15]:
```
let _ = assert (project_fst [] = [])
let _ = assert (project_fst [(1,2);(3,4);(5,6)] = [1;3;5])
```

[15]: - : unit = ()

[15]: - : unit = ()

[16]:
```
(** [project_name l] is the list of the names of the people in list [l].
    pre: [true]
    *)
    let rec project_name l = List.map (fun p -> p.name) l
```

[16]: val project_name : person list -> string list = <fun>

[85]:
```
(** [project_name l] is the list of the names of the people in list [l].
    pre: [true]
    *)
let rec project_name l =
  match l with
  | [] -> []
  | {name=n; age=_}::ps -> n::project_name ps
```

[85]:
```
val project_name : person list -> string list = <fun>
```

[17]:
```
let _ = assert (project_name [] = [])
let _ = assert (project_name [{name="a"; age=1}; {name="b"; age=2}; {name="c";␣
 ↪age=3}; {name="d"; age=4}; {name="e"; age=5}] = ["a";"b";"c";"d";"e"])
```

[17]:
```
- : unit = ()
```

[17]:
```
- : unit = ()
```

[62]:
```
(** [project_name l] is the list of the names of the people in list [l].
    pre: [true]
    *)
let project_name l =
  List.map (fun x -> x.name) l
```

[62]:
```
val project_name : person list -> string list = <fun>
```

[63]:
```
let _ = assert (project_name [] = [])
let _ = assert (project_name [{name="a"; age=1}; {name="b"; age=2}; {name="c";␣
 ↪age=3}; {name="d"; age=4}; {name="e"; age=5}] = ["a";"b";"c";"d";"e"])
```

[63]:
```
- : unit = ()
```

[63]:
```
- : unit = ()
```

4.2 Filtro

[18]:
```
(** [filter l] is the list of the names of the people in list [l].
    pre: [true]
    *)
    let rec filter p l =
      match l with
      | [] -> []
      | x::xs -> let filterN1 = filter p xs in
                 if p x
                 then x::filterN1
                 else filterN1
```

[18]:
```
val filter : ('a -> bool) -> 'a list -> 'a list = <fun>
```

[19]:
```
(** [filter l] is the list of the names of the people in list [l].
    pre: [true]
    *)
let rec filter p l =
  match l with
  | [] -> []
  | x::xs -> if p x then x::filter p xs else filter p xs
```

[19]:
```
val filter : ('a -> bool) -> 'a list -> 'a list = <fun>
```

[20]:
```
let _ = assert (filter (fun x -> x = "hello") [] = [])
let _ = assert (filter (fun x -> x = "hello") ["hello";"world";"hello";"world";
 ↪"hello"] = ["hello";"hello";"hello"])
let _ = assert (filter (fun x -> x > 3) [1;2;3;4;5] = [4;5])
let _ = assert (filter (fun x -> x < 'f') ['a';'b';'c';'d';'e';'f';'g'] = ['a';
 ↪'b';'c';'d';'e'])
```

[20]:
```
- : unit = ()
```

[20]:
```
- : unit = ()
```

[20]:
```
- : unit = ()
```

[20]:
```
- : unit = ()
```

[ ]:
```
(** [filter l] is the list of the names of the people in list [l].
    pre: [true]
    *)
    let filter p l =
      List.fold_right (fun x filterN1 -> if p x then x::filterN1 else filterN1)␣
 ↪l []
```

[89]:
```
(** [filter l] is the list of the names of the people in list [l].
    pre: [true]
    *)
let filter p l =
  List.fold_left (fun acc x -> if p x then acc@[x] else acc) [] l
```

[89]:
```
val filter : ('a -> bool) -> 'a list -> 'a list = <fun>
```

[90]:
```
let _ = assert (filter (fun x -> x = "hello") [] = [])
```

```
let _ = assert (filter (fun x -> x = "hello") ["hello";"world";"hello";"world";
 ↪"hello"] = ["hello";"hello";"hello"])
let _ = assert (filter (fun x -> x > 3) [1;2;3;4;5] = [4;5])
let _ = assert (filter (fun x -> x < 'f') ['a';'b';'c';'d';'e';'f';'g'] = ['a';
 ↪'b';'c';'d';'e'])
```

[90]:
```
- : unit = ()
```

[90]:
```
- : unit = ()
```

[90]:
```
- : unit = ()
```

[90]:
```
- : unit = ()
```

[22]:
```
(** [filter l] is the list of the names of the people in list [l].
    pre: [true]
    *)
let filter p l =
  List.rev (List.fold_left (fun acc x -> if p x then x::acc else acc) [] l)

  let filter_rev p l =
    List.fold_left (fun acc x -> if p x then x::acc else acc) [] l
```

[22]:
```
val filter : ('a -> bool) -> 'a list -> 'a list = <fun>
```

[22]:
```
val filter_rev : ('a -> bool) -> 'a list -> 'a list = <fun>
```

[ ]:
```
filter_rev (fun x -> true) [1;2;3;4;5]
```

[92]:
```
let _ = assert (filter (fun x -> x = "hello") [] = [])
let _ = assert (filter (fun x -> x = "hello") ["hello";"world";"hello";"world";
 ↪"hello"] = ["hello";"hello";"hello"])
let _ = assert (filter (fun x -> x > 3) [1;2;3;4;5] = [4;5])
let _ = assert (filter (fun x -> x < 'f') ['a';'b';'c';'d';'e';'f';'g'] = ['a';
 ↪'b';'c';'d';'e'])
```

[92]:
```
- : unit = ()
```

[92]:
```
- : unit = ()
```

[92]:
```
- : unit = ()
```

[92]:
```
- : unit = ()
```

[100]:
```
(** [filter l] is the list of the names of the people in list [l].
    pre: [true]
    *)
let filter p l =
  List.fold_right (fun x acc -> if p x then x::acc else acc) l []
```

[100]:
```
val filter : ('a -> bool) -> 'a list -> 'a list = <fun>
```

[101]:
```
let _ = assert (filter (fun x -> x = "hello") [] = [])
let _ = assert (filter (fun x -> x = "hello") ["hello";"world";"hello";"world";
 ↪"hello"] = ["hello";"hello";"hello"])
let _ = assert (filter (fun x -> x > 3) [1;2;3;4;5] = [4;5])
let _ = assert (filter (fun x -> x < 'f') ['a';'b';'c';'d';'e';'f';'g'] = ['a';
 ↪'b';'c';'d';'e'])
```

[101]:
```
- : unit = ()
```

[101]:
```
- : unit = ()
```

[101]:
```
- : unit = ()
```

[101]:
```
- : unit = ()
```

[108]:
```
(** [filter l] is the list of the names of the people in list [l].
    pre: [true]
    *)
let filter p l =
  List.fold_right (fun x xs -> x @ xs) (List.map (fun x -> if p x then [x] else␣
 ↪[]) l) []
```

[108]:
```
val filter : ('a -> bool) -> 'a list -> 'a list = <fun>
```

[109]:
```
let _ = assert (filter (fun x -> x = "hello") [] = [])
let _ = assert (filter (fun x -> x = "hello") ["hello";"world";"hello";"world";
 ↪"hello"] = ["hello";"hello";"hello"])
let _ = assert (filter (fun x -> x > 3) [1;2;3;4;5] = [4;5])
```

```
let _ = assert (filter (fun x -> x < 'f') ['a';'b';'c';'d';'e';'f';'g'] = ['a';
 ↪'b';'c';'d';'e'])
```

[109]:
```
- : unit = ()
```

[109]:
```
- : unit = ()
```

[109]:
```
- : unit = ()
```

[109]:
```
- : unit = ()
```

Há alguma versão melhor que outra? em que situações seria melhor usar cada uma delas? Quais os argumentos a usar?

4.3 Listas e pares: zip e unzip

[71]:
```
(** [zip l1 l2] is the list of pairs of elements from lists [l1] and [l2].
    pre: [length l1 = length l2]
    *)
let rec zip l1 l2 =
  match (l1, l2) with
  | ([], []) -> []
  | (x::xs, y::ys) -> (x,y)::zip xs ys
  | (_, _) -> failwith "zip: lists have different lengths"
```

[71]:
```
val zip : 'a list -> 'b list -> ('a * 'b) list = <fun>
```

[72]:
```
let _ = assert (zip [1;2;3] ['a';'b';'c'] = [(1,'a');(2,'b');(3,'c')])
```

[72]:
```
- : unit = ()
```

[76]:
```
let rec map2 f l1 l2 =
    match l1, l2 with
  | [], [] -> []
  | x::xs, y::ys -> f x y::map2 f xs ys
  | _, _ -> failwith "zip: lists have different lengths"
```

[76]:
```
val map2 : ('a -> 'b -> 'c) -> 'a list -> 'b list -> 'c list = <fun>
```

[122]:
```
(** [zip l1 l2] is the list of pairs of elements from lists [l1] and [l2].
    pre: [length l1 = length l2]
```

```
    *)
let zip l1 l2 =
  map2 (fun x y -> (x,y)) l1 l2
```

[122]:
```
val zip : 'a list -> 'b list -> ('a * 'b) list = <fun>
```

[123]:
```
let _ = assert (zip [1;2;3] ['a';'b';'c'] = [(1,'a');(2,'b');(3,'c')])
```

[123]:
```
- : unit = ()
```

[77]:
```
(** [unzip l] is the pair of lists of the first and second elements of the␣
 ↪pairs in list [l].
    pre: [true]
    *)
let rec unzip l =
  match l with
  | [] -> ([], [])
  | (x,y)::xs ->
    let (xs, ys) = unzip xs in
    (x::xs, y::ys)
```

[77]:
```
val unzip : ('a * 'b) list -> 'a list * 'b list = <fun>
```

[78]:
```
let _ = assert (unzip [] = ([], []))
let _ = assert (unzip [(1,'a');(2,'b');(3,'c')] = ([1;2;3], ['a';'b';'c']))
```

[78]:
```
- : unit = ()
```

[78]:
```
- : unit = ()
```

[93]:
```
(** [unzip l] is the pair of lists of the first and second elements of the␣
 ↪pairs in list [l].
    pre: [true]
    *)
let rec unzip l =
  List.fold_right (fun (x,y) (xs, ys) -> (x::xs, y::ys)) l ([], [])
```

[93]:
```
val unzip : ('a * 'b) list -> 'a list * 'b list = <fun>
```

[94]:
```
let _ = assert (unzip [] = ([], []))
let _ = assert (unzip [(1,'a');(2,'b');(3,'c')] = ([1;2;3], ['a';'b';'c']))
let _ = assert (unzip (zip [1;2;3] ['a';'b';'c']) = ([1;2;3], ['a';'b';'c']))
```

[94]:
```
- : unit = ()
```

[94]:
```
- : unit = ()
```

[94]:
```
- : unit = ()
```

4.4 Composição de funções

A função map pode servir também para suportar a composição de funções.

[95]:
```
let rec duplicate l =
  match l with
  | [] -> []
  | x::xs -> (2*x)::duplicate xs
```

[95]:
```
val duplicate : int list -> int list = <fun>
```

[18]:
```
let _ = assert (duplicate [1;2;3;4;5] = [2;4;6;8;10])
```

[18]:
```
- : unit = ()
```

[19]:
```
let duplicate l =
  List.map (fun x -> 2*x) l
```

[19]:
```
val duplicate : int list -> int list = <fun>
```

[21]:
```
let _ = assert (duplicate [1;2;3;4;5] = [2;4;6;8;10])
```

[21]:
```
- : unit = ()
```

[96]:
```
let string_list_of_int_list l =
  List.map string_of_int l
```

[96]:
```
val string_list_of_int_list : int list -> string list = <fun>
```

[25]:
```
let _ = assert (string_list_of_int_list [1;2;3;4;5] = ["1";"2";"3";"4";"5"])
```

[25]:
```
- : unit = ()
```

[97]:
```
let string_list_of_duplicate_int_list l = string_list_of_int_list (duplicate l)
```

[97]:
```
val string_list_of_duplicate_int_list : int list -> string list = <fun>
```

[29]:
```
let _ = assert (string_list_of_duplicate_int_list [1;2;3;4;5] = ["2";"4";"6";
 ↪"8";"10"])
```

[29]:
```
- : unit = ()
```

[32]:
```
let compose f g x = f (g x);;

let string_list_of_duplicate_int_list l = List.map (compose string_of_int (fun␣
 ↪x -> 2*x)) l
```

[32]:
```
val compose : ('a -> 'b) -> ('c -> 'a) -> 'c -> 'b = <fun>
```

[32]:
```
val string_list_of_duplicate_int_list : int list -> string list = <fun>
```

[33]:
```
let _ = assert (string_list_of_duplicate_int_list [1;2;3;4;5] = ["2";"4";"6";
 ↪"8";"10"])
```

[33]:
```
- : unit = ()
```

5 Compactar elementos

[24]:
```
(** [pack l] is the list of pairs of elements that contain the number of times␣
 ↪an element appears in a sequence [l].
   pre: [true]
   *)
let rec pack l =
  match l with
  | [] -> []
  | x::xs ->
    let tail = pack xs in
    begin match tail with
    | [] -> [(x, 1)]
    | (g, n)::gs -> if g = x then (g, n+1)::gs else (x, 1)::tail
  end
```

[24]:
```
val pack : 'a list -> ('a * int) list = <fun>
```

[26]:
```

```

```
pack ["a"; "a"; "b"; "c"; "c"; "c"; "d"; "e"; "e"; "f"; "f"; "f"; "f"; "g"; "h";
  ↪ "i"; "i"; "i"; "i"; "i"];;

let _ = assert (pack ["a"; "a"; "b"; "c"; "c"; "c"; "d"; "e"; "e"; "f"; "f";␣
  ↪"f"; "f"; "g"; "h"; "i"; "i"; "i"; "i"; "i"] = [("a", 2); ("b", 1); ("c", 3);
  ↪ ("d", 1); ("e", 2); ("f", 4); ("g", 1); ("h", 1); ("i", 5)])
```

[26]:
```
- : (string * int) list =
[("a", 2); ("b", 1); ("c", 3); ("d", 1); ("e", 2); ("f", 4); ("g", 1);
 ("h", 1); ("i", 5)]


- : unit = ()
```

[6]:
```
(** [pack l] is the list of pairs of elements that contain the number of times␣
  ↪an element appears in a sequence [l].
    pre: [true]
    *)
let pack l =
  let rec f x acc =
    match acc with
    | [] -> [(x, 1)]
    | (g, n)::gs -> if x = g then (g,n+1)::gs else (x,1)::acc
in List.fold_right f l []
```

[6]:
```
val count_by_group : 'a list -> ('a * int) list = <fun>
```

[27]:
```
pack ["a"; "a"; "b"; "c"; "c"; "c"; "d"; "e"; "e"; "f"; "f"; "f"; "f"; "g"; "h";
  ↪ "i"; "i"; "i"; "i"; "i"];;

let _ = assert (pack ["a"; "a"; "b"; "c"; "c"; "c"; "d"; "e"; "e"; "f"; "f";␣
  ↪"f"; "f"; "g"; "h"; "i"; "i"; "i"; "i"; "i"] = [("a", 2); ("b", 1); ("c", 3);
  ↪ ("d", 1); ("e", 2); ("f", 4); ("g", 1); ("h", 1); ("i", 5)])
```

[27]:
```
- : (string * int) list =
[("a", 2); ("b", 1); ("c", 3); ("d", 1); ("e", 2); ("f", 4); ("g", 1);
 ("h", 1); ("i", 5)]


- : unit = ()
```

[8]:
```
let rec unpack l =
  match l with
  | [] -> []
```

```
  | (x, n)::xs ->
    if n = 1 then x::unpack xs
    else x::unpack ((x, n-1)::xs)
```

[8]:
```
val expand_groups : ('a * int) list -> 'a list = <fun>
```

[9]:
```
let _ = assert (unpack [("a", 2); ("b", 1); ("c", 3); ("d", 1); ("e", 2); ("f",␣
 ↪4); ("g", 1); ("h", 1); ("i", 5)] =  ["a"; "a"; "b"; "c"; "c"; "c"; "d"; "e";
 ↪ "e"; "f"; "f"; "f"; "f"; "g"; "h"; "i"; "i"; "i"; "i"; "i"]);;

let l = ["a"; "a"; "b"; "c"; "c"; "c"; "d"; "e"; "e"; "f"; "f"; "f"; "f"; "g";␣
 ↪"h"; "i"; "i"; "i"; "i"; "i"] in
  assert (unpack (pack l) = l)
```

[9]:
```
- : unit = ()
```

[9]:
```
- : unit = ()
```

5.1 Agrupar elementos

[10]:
```
let rec group_by l =
  match l with
  | [] -> []
  | x::xs ->
    let gs = group_by xs in
    begin match gs with
    | [] -> [[x]]
    | g::gs ->
      begin match g with
      | [] -> assert false (* cannot be empty*)
      | y::_ -> if y = x then (x::g)::gs else [x]::g::gs
      end
    end
```

[10]:
```
val group_by : 'a list -> 'a list list = <fun>
```

[11]:
```
let _ = assert (group_by [1; 1; 2; 3; 3; 3; 4; 5; 5; 6; 6; 6; 6; 7; 8; 9; 9; 9;␣
 ↪9; 9] = [[1; 1]; [2]; [3; 3; 3]; [4]; [5; 5]; [6; 6; 6; 6]; [7]; [8]; [9; 9;␣
 ↪9; 9; 9]])
```

[11]:
```
- : unit = ()
```

[6]:
```
let group_by l =
  let rec f x acc =
    match acc with
    | [] -> [[ x ]]
    | g::gs ->
      begin match g with
      | [] -> assert false (* all lists have elements, see above *)
      | y::ys -> if x = y then (x::y::ys)::gs else [ x ]::acc
      end
  in List.fold_right f l []
```

[6]:
```
val group_by : 'a list -> 'a list list = <fun>
```

[13]:
```
let _ = assert (group_by [1; 1; 2; 3; 3; 3; 4; 5; 5; 6; 6; 6; 6; 7; 8; 9; 9; 9;␣
↪9; 9] = [[1; 1]; [2]; [3; 3; 3]; [4]; [5; 5]; [6; 6; 6; 6]; [7]; [8]; [9; 9;␣
↪9; 9; 9]])
```

[13]:
```
- : unit = ()
```

6 Desagrupar elementos

[15]:
```
let rec ungroup l =
  match l with
  | [] -> []
  | []::gs -> ungroup gs
  | (x::xs)::gs -> x::ungroup (xs::gs)
```

[15]:
```
val ungroup : 'a list list -> 'a list = <fun>
```

[23]:
```
let _ = assert (ungroup [[1; 1]; [2]; [3; 3; 3]; [4]; [5; 5]; [6; 6; 6; 6]; [7];
↪ [8]; [9; 9; 9; 9; 9]] = [1; 1; 2; 3; 3; 3; 4; 5; 5; 6; 6; 6; 6; 7; 8; 9; 9;␣
↪9; 9; 9]);;

let l = [1; 1; 2; 3; 3; 3; 4; 5; 5; 6; 6; 6; 6; 7; 8; 9; 9; 9; 9; 9] in assert␣
↪(ungroup (group_by l) = l)
```

[23]:
```
- : unit = ()
```

[23]:
```
- : unit = ()
```

[22]:
```
let ungroup gs = List.fold_right (fun xs ys -> (List.fold_right (fun x ys -> x :
↪: ys) xs ys)) gs []
```

[22]:
```
val ungroup : 'a list list -> 'a list = <fun>
```

[17]:
```
let ungroup gs = List.fold_right (@) gs []
```

[17]:
```
val ungroup : 'a list list -> 'a list = <fun>
```

[4]:
```
let ungroup xs = List.concat xs
```

[4]:
```
val ungroup : 'a list list -> 'a list = <fun>
```

[18]:
```
let l = [1; 1; 2; 3; 3; 3; 4; 5; 5; 6; 6; 6; 6; 7; 8; 9; 9; 9; 9; 9] in assert␣
 ↪(ungroup (group_by l) = l)
```

[18]:
```
- : unit = ()
```

7 merge de duas listas ordenadas

[40]:
```
let rec merge l1 l2 =
  match l1, l2 with
  | [], [] -> []
  | x::xs, [] -> l1
  | [], y::ys -> l2
  | x::xs, y::ys -> if x < y then x::merge xs l2 else y::merge l1 ys
```

[40]:
```
val merge : 'a list -> 'a list -> 'a list = <fun>
```

[41]:
```
let _ = assert (merge [1;3;5;7;9] [2;4;6;8;10] = [1;2;3;4;5;6;7;8;9;10])
```

[41]:
```
- : unit = ()
```

8 merge sort